

Projet Final — Développer pour le Cloud

(Master 2) · v3 LOCAL-FIRST

Séances 7-8-9 | YNOV Campus Montpellier | 2025-2026

Intervenant : M. Benhamdi Ayoub **Dates :** J7 = 25 + 26 juin 2026 · J8 = 30 juin + 1^{er} juillet 2026 · J9 = 6 juillet 2026 (soutenance) **Durée totale : 18h de production en groupe + 3h de soutenance = 21h**
Plateforme : Stack 100% locale, open-source, zéro carte bancaire (cf. `GUIDE_SETUP_LOCAL.md`)

v3 — juin 2026 : version pivotée vers une stack **100% locale** suite à la coupure des crédits cloud étudiants. Mêmes 9 blocs obligatoires que la v2. Niveau d'ambition **identique**. Les concepts cloud-native sont préservés via leurs équivalents open-source : Kubernetes local (kind/k3d), GitHub Container Registry, Redpanda/RabbitMQ, HashiCorp Vault, ArgoCD, Knative, Kubecost.

1. Préambule

Vous avez parcouru pendant six séances l'ensemble du cycle de vie d'une application *cloud-native*. **Les concepts ne changent pas** — seuls les *runtimes* changent. Ce qui était managé par GCP devient *self-hosted* dans votre cluster Kubernetes local.

Séance	Compétence pratiquée	Outil utilisé en projet final v3
J1	Fondamentaux IAM, conteneurisation, registry	RBAC K8s, Docker, GitHub Container Registry (ghcr.io)
J2	Docker multi-stage, runtime serverless, VPC	Multi-stage Dockerfile, Knative Serving (équiv. Cloud Run), Network Policies
J3	Orchestrateur K8s, IaC, CI/CD	kind ou k3d (cluster local), Terraform providers Kubernetes/Helm , GitHub Actions
J4	DevSecOps, secrets, FinOps, event-driven	Trivy local, HashiCorp Vault + External Secrets Operator , Kubecost , Knative Eventing
J5	Messaging avancé, NoSQL, service mesh	Redpanda (Kafka-compatible) OU RabbitMQ OU NATS ; MongoDB / PostgreSQL ; Linkerd ou Istio
J6	GitOps, observabilité, SRE, chaos	ArgoCD App of Apps , kube-prometheus-stack , Loki + Promtail , Tempo + OpenTelemetry , Chaos Mesh

Le projet final est l'**unique évaluation sommative** du module. Il consiste à concevoir, déployer et soutenir une **architecture cloud-native complète sur un cluster Kubernetes local** qui mobilise un sous-ensemble cohérent de ces compétences.

⚠ Le projet n'est **pas** la réécriture des TP1-TP6. C'est un projet **autonome**, avec un produit logiciel **inédit** dont vous êtes la *Platform & Software Engineering team*. Vous pouvez réutiliser du code, des modules ou des manifestes des TP comme **briques**, mais l'agrégation, l'intégration et la cohérence d'ensemble sont au cœur de l'évaluation.

💡 **Pourquoi local-first ?** Les compétences cloud-native sont **portables** : un déploiement ArgoCD sur kind se transpose en 30 minutes sur GKE/EKS/AKS le jour où vous aurez des crédits. Vous démontrez **la maîtrise des concepts**, pas la dépendance à un fournisseur.

2. Modalités

- **Groupes** : 2 à 3 apprenants (1 personne seule autorisée à titre exceptionnel, sur validation formateur).
- **Choix du contexte métier** : libre, parmi les 4 sujets proposés en §3 ou un sujet personnel validé par le formateur en début de J7.
- **Dépôt Git** : un dépôt par groupe (GitHub **public obligatoire** pour profiter du quota CI/CD gratuit illimité + GHCR illimité), accès lecture pour le formateur (`abconsulting113@gmail.com`).
- **Cluster Kubernetes** : 1 cluster local **par groupe** (au moins une machine du groupe avec **≥ 16 Go RAM** recommandé, **8 Go minimum** avec stack allégée — cf. `GUIDE_SETUP_LOCAL.md` §1).
- **Coût** : **0 €** — la stack est 100% locale et open-source.
- **Soutenance** : **25 minutes par groupe** (cf. §6) le **6 juillet 2026**.

- ⚠ **Démo soutenance** : votre cluster tournera sur le portable d'un membre du groupe. Prévoyez :
- Chargeur secteur (la batterie ne tient pas 25 min sous Prometheus + Loki + Linkerd).
 - Connexion filaire si possible (Wi-Fi YNOV peut être chargé).
 - **Plan B** : vidéo enregistrée de la démo (60-90s) au cas où le cluster crashe en live.

3. Sujets proposés (choisir UN seul)

Les 4 sujets sont **identiques à la v2** — l'enjeu métier ne change pas, seules les briques d'infrastructure changent.

🟢 **Sujet A — GreenLogistics : tracking temps réel de livraison dernière mile**

Pitch : Startup de livraison écologique. Tracker des colis en temps réel, optimiser les tournées, dashboard client.

Domaines fonctionnels minimum :

1. **API de gestion de colis** (REST/HTTP) — création, statuts, recherche.
2. **Service d'ingestion de positions GPS** (événementiel) — un point GPS toutes les 5 secondes par livreur (simulé).

3. **Service de notification** — événement « livraison à 5 min » au destinataire.
4. **Front simple ou API publique** pour suivre un colis.

● **Sujet B — *MediConnect* : plateforme de télésuivi de patients chroniques**

Pitch : SaaS B2B pour cabinets médicaux. Mesures patient (tension, glycémie, poids), alertes médecin. Exigences RGPD fortes.

Domaines fonctionnels minimum :

1. **API d'enregistrement de mesures** (authentifiée).
2. **Moteur de règles** : détection d'anomalies (asynchrone, événementiel).
3. **Service d'alertes** (email simulé via MailHog OU webhook).
4. **Dashboard médecin** ou API consolidée.

● **Sujet C — *EduStream* : plateforme de cours en ligne avec quiz temps réel**

Pitch : LMS startup. Diffusion live, quiz en temps réel, dashboard statistique. Pics de charge attendus.

Domaines fonctionnels minimum :

1. **API de gestion des cours / quiz** (CRUD).
2. **Service de collecte de réponses** (haute fréquence, événementiel).
3. **Service d'agrégation statistique** (fenêtres glissantes).
4. **API de restitution** pour l'enseignant.

● **Sujet D — *EcoEnergy* : supervision IoT de capteurs photovoltaïques**

Pitch : Opérateur de centrales solaires. Capteurs envoyant données toutes les 30s. Détection de panne, prévision de production.

Domaines fonctionnels minimum :

1. **Endpoint d'ingestion** (HTTP ou messaging direct).
2. **Service de détection de pannes** (règles + seuils).
3. **Service de prévision** simplifié (moyenne mobile suffit).
4. **API/Dashboard** d'exploitation.

💡 **Sujet libre** : tout autre contexte avec ≥ 3 microservices, communication asynchrone, persistance.
À valider **avant le J7**.

4. Exigences techniques (toutes obligatoires)

Mêmes 9 blocs que la v2, transposés sur stack open-source locale.

4.1 — Architecture & conception (25 pts)

Exigence	Outil v3 (local)	Outil v2 (GCP) — pour info
----------	------------------	-------------------------------

≥ 3 microservices distincts dans leur propre conteneur	Pods K8s sur kind/k3d	GKE Autopilot
Communication asynchrone entre ≥ 2 services : 1 topic + 1 souscription minimum, Dead Letter recommandé	Redpanda (reco) OU RabbitMQ OU NATS JetStream	Cloud Pub/Sub
Persistance managée : au moins une base de données	PostgreSQL (StatefulSet) OU MongoDB OU ClickHouse (analytique, bonus)	Cloud SQL / Firestore / BigQuery
Diagramme d'architecture committé	draw.io / Excalidraw / Mermaid	idem
ADR.md : 3 à 6 ADR (Contexte / Décision / Conséquences)	—	idem

4.2 — Conteneurisation & runtime (15 pts)

Exigence	Outil v3	Outil v2 (GCP)
Dockerfile multi-stage par service, image < 250 Mo	Docker, Distroless / Alpine	idem
Images publiées avec tags sémantiques	GitHub Container Registry (ghcr.io)	Artifact Registry
Déploiement Kubernetes majoritaire. Knative Serving accepté pour 1 service (équiv. Cloud Run)	kind ou k3d (≥ 3 nodes)	GKE Autopilot
HPA sur le service le plus sollicité	metrics-server + HPA standard	idem
Resources requests / limits sur tous les pods	idem	idem

4.3 — Infrastructure-as-Code (10 pts)

Exigence	Outil v3	Outil v2 (GCP)
Terraform pour provisionner : cluster local, namespaces, ArgoCD, Vault, Redpanda, monitoring	Providers tehcyrx/kind OU alekc/k3d , hashicorp/kubernetes , hashicorp/helm	Provider google
State Terraform versionné (local OK : state/terraform.tfstate committé dans une branche dédiée OU remote state sur S3-compatible MinIO local)	MinIO ou state local	Bucket GCS versionné
Au moins 1 module Terraform réutilisable maison	—	idem

4.4 — CI/CD sécurisé (15 pts)

Exigence	Outil v3	Outil v2 (GCP)
Pipeline GitHub Actions : lint + tests unitaires + build + push GHCR + scan CVE Trivy (échec si CRITICAL) + déploiement	GitHub Actions (gratuit illimité repo public)	idem
Authentification keyless GitHub Actions → cluster	OIDC GitHub Actions → kubeconfig court-vivant (via <i>webhook custom OU service account token rotated</i>) — OU clone du repo gitops par ArgoCD (pas besoin de pousser sur le cluster depuis CI)	Workload Identity Federation
Secrets stockés hors Git	HashiCorp Vault (dev-mode) + External Secrets Operator synchronisant vers K8s Secrets	Secret Manager

💡 **Pattern recommandé** : la CI ne touche **pas** le cluster. Elle pousse l'image sur GHCR + met à jour un tag dans le repo `*-gitops` . **ArgoCD** (qui tourne dans votre cluster) tire le changement et déploie. C'est le pattern « pull-based » du TP6, et il évite tout le casse-tête d'authentification CI → cluster local.

4.5 — GitOps & déploiement progressif (10 pts)

Exigence	Outil v3	Outil v2 (GCP)
ArgoCD installé sur le cluster, syncro d'un dépôt <code>*-gitops</code> (App of Apps recommandé)	idem	idem
Self-Heal activé et démontré en live	idem	idem
Canary OU Blue/Green sur ≥ 1 service : Argo Rollouts + AnalysisTemplate (reco — TP6) OU Linkerd SMI TrafficSplit OU Istio VirtualService	Argo Rollouts OU Istio	idem

4.6 — Observabilité & SRE (15 pts)

Exigence	Outil v3	Outil v2 (GCP)
kube-prometheus-stack déployé (Prometheus + Grafana)	idem	idem
Loki + Promtail pour logs centralisés	idem	idem
<code>/metrics</code> Prometheus exposé sur ≥ 1 service	idem	idem
≥ 2 SLO définis avec Recording Rules Prometheus	idem	idem
Dashboard Grafana custom : taux d'erreur, latence P95/P99, Error Budget	idem	idem
≥ 1 alerte Prometheus active (Alertmanager → webhook MailHog OU smtp4dev OU Discord webhook)	Alertmanager + MailHog local	Alertmanager + email

4.7 — Sécurité (DevSecOps) (5 pts)

Exigence	Outil v3	Outil v2 (GCP)
Image Trivy sans CVE HIGH/CRITICAL non justifiée	idem	idem
Network Policies Kubernetes (deny-by-default ≥ 1 namespace)	Cilium (kind avec CNI Cilium) OU Calico	Calico (Autopilot)
mTLS automatique pour le trafic intra-cluster ≥ 1 namespace : Linkerd (reco, mTLS auto) OU Istio (plus lourd) OU NGINX Ingress + cert-manager pour la passerelle externe	Istio / Cloud Armor / IAP	
Identité service-à-service : SPIFFE/SPIRE (bonus) OU K8s ServiceAccount + RoleBinding minimum	Workload Identity GCP	

4.8 — FinOps / RessourceOps (5 pts)

Sur cluster local, le « coût » devient **la consommation CPU/RAM** — la même logique de gouvernance, sans facture.

Exigence	Outil v3	Outil v2 (GCP)
Kubecost (Helm chart open-source) déployé, dashboard accessible	Kubecost	Cloud Billing
Labels / Annotations (team , env , app) sur toutes les ressources K8s	metadata.labels standard	Labels GCP
Capture du rapport Kubecost filtré par namespace dans le rapport technique	Kubecost UI	Billing GCP
Estimation du coût mensuel projeté en production GCP/AWS/Azure (Kubecost donne une projection automatique)	Kubecost forecast	Kubecost / cloud calculator

4.9 — Bonus (5 pts max, plafonnés)

Bonus	Pts	Outil v3
Chaos Engineering : 1 expérience Chaos Mesh (PodChaos ou NetworkChaos) + rapport <code>CHAOS_REPORT.md</code>	+2	Chaos Mesh
Tracing distribué Tempo + OpenTelemetry bout-en-bout (trace_id corrélé logs $\leftrightarrow$ traces dans Grafana)	+2	Tempo + OTEL
Runbook DR théorique : RTO/RPO chiffrés, plan de bascule cluster $\rightarrow$ cloud public	+2	— (théorique)
Knative Eventing : event-driven function déclenchée par votre broker (équivalent Cloud Function)	+1	Knative
Suite e2e (Cypress, Playwright, k6) intégrée au pipeline CI	+1	—

SPIFFE/SPIRE : identité workload portable (équivalent Workload Identity Federation)	+1	SPIRE
--	----	-------

5. Livrables

À déposer dans le dépôt Git **avant le lundi 6 juillet 2026 à 09h00** :

1. **Code source** de tous les services + manifestes K8s + Terraform.
2. **README.md** racine (voir `TEMPLATE_README_projet.md` — adapter les URL `gcloud` → équivalents locaux).
3. **ADR.md** : 3 à 6 décisions architecturales.
4. **RAPPORT_TECHNIQUE.md** (voir `TEMPLATE_Rapport_Technique.md`) : 8 à 15 pages.
5. **Diagramme d'architecture** (`docs/architecture.png` ou Mermaid).
6. **Captures d'écran** dans `docs/captures/` : ArgoCD synced, dashboard Grafana, pipeline CI vert, **dashboard Kubecost** par namespace.
7. **Slides de soutenance** (`docs/soutenance.pdf` ou lien public).
8. **SETUP.md** ou `Makefile` : commandes pour qu'**un correcteur extérieur reproduise votre cluster en < 30 min** sur sa machine.

Tag git de la version soutenue : `v1.0-soutenance` .

6. Soutenance (lundi 6 juillet 2026)

Format : 25 minutes par groupe :

Minutes	Étape	Conseil
0-3	Pitch & contexte métier	Persona, KPI métier.
3-8	Architecture cible	Diagramme, choix techniques, alternatives écartées.
8-15	Démo live	Pipeline qui déploie, ArgoCD sync, Grafana en temps réel, Self-Heal <i>ou</i> Canary.
15-20	Retour d'expérience	Ce qui a marché, ce qui a coincé. Honnêteté valorisée.
20-25	Questions du jury	Préparez des réponses sur les choix non implémentés.

Tous les membres prennent la parole. Modulation individuelle ± 3 pts si déséquilibre flagrant.

💡 **Démo live sur cluster local :**

- Démarrez **kind/k3d en début de séance** (le matin) — pas au moment de la démo.
- Pré-tirez les images sur la machine de démo (`docker pull` la veille).
- Préparez `kubect1 port-forward` ouverts à l'avance dans plusieurs terminaux.

7. Calendrier détaillé

Date	Heure	Phase	Livrable de fin de séance
jeu. 25 juin	10h-13h	J7-1 Kickoff : choix sujet, groupes, archi v0, kind/k3d installé sur la machine principale	Diagramme v0 + repo + cluster local UP
ven. 26 juin	09h-13h	J7-2 Build : Dockerfile, Terraform (cluster + namespaces), Redpanda déployé, 1 ^{re} image sur GHCR	1 service Running sur le cluster local
mar. 30 juin	09h-13h	J8-1 Intégration : pipeline GitHub Actions → GHCR, ArgoCD installé, intégration async entre 2 services	Pipeline vert + ArgoCD synced + Pub/Sub-like fonctionnel
mer. 1 ^{er} juil.	14h-17h	J8-2 Observabilité : kube-prometheus-stack, Loki, SLO, Network Policies, Kubecost	Dashboard SLO + Kubecost opérationnels
lun. 6 juil.	09h-13h	J9-1 Finitions : Canary, bonus, rapport, slides, code freeze	Code freeze + slides prêts
lun. 6 juil.	14h-17h	J9-2 Soutenances	Notes individuelles

⚠ **Capacité jury** : 17 inscrits → **6 groupes** maximum dans le créneau 14h-17h (25 min/groupe). Au-delà : 2 sous-jurys parallèles OU allongement jusqu'à 17h30.

Voir `PLANNING_J7-J8-J9.md` pour le détail.

8. Critères d'évaluation

Critère	Pondération
Architecture & conception	25 pts
Implémentation & qualité du code	25 pts
Pipeline CI/CD & GitOps	15 pts
Observabilité & SLO	15 pts
Sécurité (DevSecOps)	10 pts
Documentation & README	10 pts
Total	100 pts
Bonus (cf. §4.9)	+5 pts plafonnés

9. Règles de fair-play

- **Plagiat** = 0/100 pour tous les membres. Citez vos sources (Helm charts, modules Terraform, Stack Overflow, docs officielles) en bibliographie.
- **IA générative** autorisée pour boilerplate / debug / doc. **Vous êtes responsables** de chaque ligne livrée.
- **Commits** : ≥ 1 /jour/membre actif. Repo monocommit final = perte de points en `Documentation`.

10. Support pendant les séances

- En présentiel pendant J7, J8, J9 — le formateur passe sur chaque groupe.
- Hors créneaux : canal Discord/Slack du module (réponse sous 24h ouvrées).
- `GUIDE_SETUP_LOCAL.md` : reference technique commune. Pas de support pour ce qui est déjà documenté dedans.

11. Garde-fous opérationnels

- **J7-1** : ≤ 4 microservices au diagramme. Au-delà, vous ne finirez pas.
- **J7-2** : si votre cluster n'a pas Redpanda + 1 service `Running` à 13h, **escalade au formateur**.
- **J8-1** : Trivy doit faire `exit 1` sur CVE `CRITICAL`. Si `continue-on-error: true` est présent, le critère 4.4 chute.
- **J8-2** : si la machine de démo dépasse **85% RAM** avec la stack complète, désactivez Tempo (passez-le en bonus J9 ou supprimez-le).
- **J9-1** : aucun bonus tenté après 11h00 (pas le temps d'observer les effets). Stabilisez la démo.
- **J9-2** : après votre démo, lancez `kind delete cluster` ou `k3d cluster delete` devant le jury → **+1 pt « FinOps locale »** (geste de propreté).

12. Stack technique de référence (résumé en un tableau)

Domaine	Outil v3 par défaut	Alternatives acceptées
Cluster K8s	kind	k3d, minikube
Registry	GitHub Container Registry (ghcr.io)	Harbor, registry:2 local
CI/CD	GitHub Actions (repo public)	GitLab CI, Forgejo
GitOps	ArgoCD	Flux
Messaging	Redpanda	RabbitMQ, NATS JetStream
Persistence SQL	PostgreSQL (StatefulSet)	MariaDB
Persistence NoSQL	MongoDB (StatefulSet)	CouchDB

Analytique (bonus)	ClickHouse	DuckDB
Secrets	Vault dev-mode + External Secrets Operator	Sealed Secrets, SOPS
Service Mesh	Linkerd (<i>reco, léger</i>)	Istio (<i>plus lourd</i>)
Identité workload	K8s SA + RoleBinding	SPIFFE/SPIRE (<i>bonus</i>)
Functions event-driven (bonus)	Knative Eventing	OpenFaaS
Métriques	kube-prometheus-stack	—
Logs	Loki + Promtail	Fluent Bit + Elasticsearch (<i>lourd</i>)
Traces (bonus)	Tempo + OpenTelemetry	Jaeger
Chaos (bonus)	Chaos Mesh	Litmus
FinOps	Kubecost	OpenCost
Email simulé (alertes)	MailHog OU smtp4dev	webhook Discord/Slack

Toutes ces briques sont **gratuites, open-source**, et fonctionnent sur **kind / k3d**. Voir `GUIDE_SETUP_LOCAL.md` pour les commandes d'installation détaillées.

Bon projet, et **amusez-vous** — c'est aussi ça, faire du cloud.

— A. Benhamdi, juin 2026 (v3 LOCAL-FIRST)